

Task 11: Thinking in React

Summary

In this task, I learned the recommended approach for building React applications by following the "Thinking in React" methodology. Instead of directly writing code, I learned how to analyze a UI design, break it into smaller reusable components, and organize them into a clear component hierarchy. This approach makes applications easier to understand, develop, and maintain.

I learned that the first step in building a React application is to divide the UI into independent components based on their responsibilities. After creating the component hierarchy, the next step is to build a static version of the application using props to pass data from parent components to child components without adding any interactivity. This helps ensure that the application's structure is correct before introducing state and user interactions.

Another important concept I learned was identifying the minimal but complete state required for an application. React encourages storing only the data that changes over time while calculating all other values whenever needed. This avoids redundant state, reduces bugs, and keeps components simple and efficient.

I also understood how to decide where state should live within the component tree. React follows one-way data flow, meaning data moves from parent components to child components through props. When multiple components require the same state, the state should be placed in their closest common parent and shared through props. This ensures that all components remain synchronized and maintain a single source of truth.

Finally, I learned the complete five-step process of building React applications: breaking the UI into components, creating a static UI, identifying the minimal state, placing the state in the correct component, and adding inverse data flow so child components can update parent state through event handlers. This structured approach helps build scalable, reusable, and maintainable React applications.

Learning Outcome

After completing this task, I understood how to think like a React developer by planning applications before coding, creating reusable component hierarchies, identifying the minimum required state, managing one-way data flow, and organizing state effectively. These concepts provide a strong foundation for building clean, scalable, and maintainable React applications.

Task 12: React Router

Summary

In this task, I learned how to implement client-side routing in React applications using React Router. I understood that React Router allows users to navigate between different pages without reloading the entire application, creating a faster and smoother Single Page Application (SPA) experience. Instead of requesting a new page from the server for every navigation, React Router updates only the required components on the client side.

I learned how to install and configure React Router by setting up a router and wrapping the application with the routing provider. I understood the purpose of routing components such as BrowserRouter (or the newer router provider approach), Routes, and Route, which define different URLs and specify which component should be displayed for each path. This forms the foundation of navigation in React applications.

Another important concept I learned was creating multiple pages and navigating between them using the Link component instead of traditional HTML anchor tags. The Link component allows navigation without refreshing the page, making the application faster and preserving the current application state. I also explored nested routes and layout components, which help organize large applications by sharing common UI elements such as navigation bars, headers, and footers across multiple pages.

I also learned how React Router supports dynamic routing, URL parameters, navigation between pages, and handling invalid URLs using a "Not Found" route. These features make it possible to build modern applications such as dashboards, e-commerce websites, blogs, and management systems with structured and user-friendly navigation.

Finally, I understood that React Router separates navigation logic from UI components, making applications more modular and easier to maintain. By organizing routes properly and using reusable page components, developers can build scalable React applications with clean navigation and an improved user experience.

Learning Outcome

After completing this task, I understood how to implement client-side routing in React applications using React Router, configure routes, navigate between pages without reloading the browser, create nested routes and layouts, and manage dynamic navigation efficiently. These concepts are essential for building modern, scalable, and user-friendly React Single Page Applications (SPAs).

⇒ **GITHUB REPO LINK:**

https://github.com/yashrathod910/15D_Internship/tree/main/DAY7

